

COMPUTER NETWORKS

Practical Programs - Complete Collection

19 Complete Programs

Generated on May 07, 2026

Question 1: TCP Server Program

SERVER: TCPServer.java

```
import java.io.*;
import java.net.*;

public class TCPServer {
    public static void main(String[] args) {
        try {
            ServerSocket serverSocket = new ServerSocket(8080);
            System.out.println("Server is listening...");
            Socket socket = serverSocket.accept();
            BufferedReader in = new BufferedReader(
                new InputStreamReader(socket.getInputStream()));
            String message = in.readLine();
            System.out.println("Message from client: " + message);
            socket.close();
            serverSocket.close();
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

CLIENT: TCPClient.java

```
import java.io.*;
import java.net.*;

public class TCPClient {
    public static void main(String[] args) {
        try {
            Socket socket = new Socket("localhost", 8080);
            PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
            String message = "hello_client";
            out.println(message);
            System.out.println("Message sent: " + message);
            socket.close();
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

Question 2: UDP Echo Client and Server

SERVER: UDPEchoServer.java

```
import java.net.*;

public class UDPEchoServer {
    public static void main(String[] args) {
        try {
            DatagramSocket socket = new DatagramSocket(9876);
            byte[] receiveData = new byte[1024];
            System.out.println("UDP Server is running...");
            DatagramPacket receivePacket = new DatagramPacket(
                receiveData, receiveData.length);
            socket.receive(receivePacket);
            String message = new String(receivePacket.getData(), 0,
                receivePacket.getLength());
            System.out.println("Received from client: " + message);
            InetAddress clientAddress = receivePacket.getAddress();
            int clientPort = receivePacket.getPort();
            byte[] sendData = message.getBytes();
            DatagramPacket sendPacket = new DatagramPacket(sendData,
                sendData.length, clientAddress, clientPort);
            socket.send(sendPacket);
            socket.close();
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

CLIENT: UDPEchoClient.java

```
import java.net.*;

public class UDPEchoClient {
    public static void main(String[] args) {
        try {
            DatagramSocket socket = new DatagramSocket();
            String message = "hello_client";
            byte[] sendData = message.getBytes();
            byte[] receiveData = new byte[1024];
            InetAddress serverAddress =
                InetAddress.getByName("localhost");
            DatagramPacket sendPacket = new DatagramPacket(sendData,
                sendData.length, serverAddress, 9876);
            socket.send(sendPacket);
            DatagramPacket receivePacket = new DatagramPacket(
                receiveData, receiveData.length);
            socket.receive(receivePacket);
            String receivedMessage = new String(receivePacket.getData(), 0,
                receivePacket.getLength());
            System.out.println("Sent: " + message);
            System.out.println("Received: " + receivedMessage);
            socket.close();
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

Question 3: Chat Application using TCP

SERVER: ChatServer.java

```
import java.io.*;
import java.net.*;

public class ChatServer {
    public static void main(String[] args) {
        try {
            ServerSocket serverSocket = new ServerSocket(1234);
            System.out.println("Server waiting...");
            Socket socket = serverSocket.accept();
            System.out.println("Client connected!");
            BufferedReader in = new BufferedReader(
                new InputStreamReader(socket.getInputStream()));
            PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
            BufferedReader userInput = new BufferedReader(
                new InputStreamReader(System.in));
            String clientMsg, serverMsg;
            while (true) {
                clientMsg = in.readLine();
                System.out.println("Client: " + clientMsg);
                if (clientMsg.equalsIgnoreCase("exit")) break;
                System.out.print("Server: ");
                serverMsg = userInput.readLine();
                out.println(serverMsg);
                if (serverMsg.equalsIgnoreCase("exit")) break;
            }
            socket.close();
            serverSocket.close();
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

CLIENT: ChatClient.java

```
import java.io.*;
import java.net.*;

public class ChatClient {
    public static void main(String[] args) {
        try {
            Socket socket = new Socket("localhost", 1234);
            System.out.println("Connected to server!");
            BufferedReader in = new BufferedReader(
                new InputStreamReader(socket.getInputStream()));
            PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
            BufferedReader userInput = new BufferedReader(
                new InputStreamReader(System.in));
            String clientMsg, serverMsg;
            while (true) {
                System.out.print("Client: ");
                clientMsg = userInput.readLine();
                out.println(clientMsg);
                if (clientMsg.equalsIgnoreCase("exit")) break;
                serverMsg = in.readLine();
                System.out.println("Server: " + serverMsg);
                if (serverMsg.equalsIgnoreCase("exit")) break;
            }
            socket.close();
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

Question 4: DNS Implementation using UDP

SERVER: DNSServer.java

```
import java.net.*;

public class DNSServer {
    public static void main(String[] args) {
        try {
            DatagramSocket socket = new DatagramSocket(9876);
            byte[] receiveData = new byte[1024];
            System.out.println("DNS Server is running...");
            StringBuilder domainBuilder = new StringBuilder();
            InetAddress clientAddress = null;
            int clientPort = 0;
            while (true) {
                DatagramPacket packet = new DatagramPacket(receiveData,
                    receiveData.length);
                socket.receive(packet);
                String part = new String(packet.getData(), 0,
                    packet.getLength());
                if (part.equals("END")) {
                    clientAddress = packet.getAddress();
                    clientPort = packet.getPort();
                    break;
                }
                domainBuilder.append(part);
            }
            String domain = domainBuilder.toString();
            System.out.println("Received domain: " + domain);
            InetAddress address = InetAddress.getByName(domain);
            String ip = address.getHostAddress();
            byte[] sendData = ip.getBytes();
            DatagramPacket sendPacket = new DatagramPacket(sendData,
                sendData.length, clientAddress, clientPort);
            socket.send(sendPacket);
            socket.close();
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

CLIENT: DNSClient.java

```
import java.net.*;
import java.util.Scanner;

public class DNSClient {
    public static void main(String[] args) {
        try {
            DatagramSocket socket = new DatagramSocket();
            Scanner sc = new Scanner(System.in);
            System.out.print("Enter domain name: ");
            String domain = sc.nextLine();
            System.out.print("Enter frame size: ");
            int size = sc.nextInt();
            InetAddress serverAddress =
                InetAddress.getByName("localhost");
            for (int i = 0; i < domain.length(); i += size) {
                String frame = domain.substring(i,
                    Math.min(i + size, domain.length()));
                byte[] sendData = frame.getBytes();
                DatagramPacket packet = new DatagramPacket(sendData,
                    sendData.length, serverAddress, 9876);
                socket.send(packet);
            }
            byte[] endSignal = "END".getBytes();
            DatagramPacket endPacket = new DatagramPacket(endSignal,
                endSignal.length, serverAddress, 9876);
            socket.send(endPacket);
            byte[] receiveData = new byte[1024];
            DatagramPacket receivePacket = new DatagramPacket(receiveData,
                receiveData.length);
            socket.receive(receivePacket);
            String ip = new String(receivePacket.getData(), 0,
                receivePacket.getLength());
            System.out.println("Resolved IP: " + ip);
            socket.close();
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

Question 5a: File Transfer using TCP

SERVER: FileServer.java

```
import java.io.*;
import java.net.*;

public class FileServer {
    public static void main(String[] args) {
        try {
            ServerSocket serverSocket = new ServerSocket(5000);
            System.out.println("Server waiting...");
            Socket socket = serverSocket.accept();
            System.out.println("Client connected");
            InputStream in = socket.getInputStream();
            FileOutputStream fos = new FileOutputStream("received.txt");
            byte[] buffer = new byte[1024];
            int count;
            while ((count = in.read(buffer)) > 0) {
                fos.write(buffer, 0, count);
            }
            System.out.println("File received successfully");
            fos.close();
            socket.close();
            serverSocket.close();
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

CLIENT: FileClient.java

```
import java.io.*;
import java.net.*;

public class FileClient {
    public static void main(String[] args) {
        try {
            Socket socket = new Socket("localhost", 5000);
            FileInputStream fis = new FileInputStream("send.txt");
            OutputStream out = socket.getOutputStream();
            byte[] buffer = new byte[1024];
            int count;
            while ((count = fis.read(buffer)) > 0) {
                out.write(buffer, 0, count);
            }
            System.out.println("File sent successfully");
            fis.close();
            socket.close();
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

Question 6: ARP Protocol Simulation

```
import java.util.*;

public class ARPSimulation {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        HashMap<String, String> arpTable = new HashMap<>();
        arpTable.put("192.168.1.1", "AA:BB:CC:DD:EE:01");
        arpTable.put("192.168.1.2", "AA:BB:CC:DD:EE:02");
        arpTable.put("192.168.1.3", "AA:BB:CC:DD:EE:03");
        System.out.print("Enter IP address: ");
        String ip = sc.nextLine();
        if (arpTable.containsKey(ip)) {
            System.out.println("MAC Address: " + arpTable.get(ip));
        } else {
            System.out.println("IP not found in ARP table");
        }
        sc.close();
    }
}
```

Question 7: RARP Protocol using UDP

```
import java.net.*;
import java.util.HashMap;
import java.util.Scanner;

public class RARPServer {
    public static void main(String[] args) {
        try {
            DatagramSocket socket = new DatagramSocket(9876);
            HashMap<String, String> rarpTable = new HashMap<>();
            rarpTable.put("AA:BB:CC:DD:EE:01", "192.168.1.1");
            System.out.println("RARP Server running...");
            byte[] receiveData = new byte[1024];
            DatagramPacket receivePacket = new DatagramPacket(
                receiveData, receiveData.length);
            socket.receive(receivePacket);
            String mac = new String(receivePacket.getData(), 0,
                receivePacket.getLength());
            String ip = rarpTable.getDefault(mac, "IP not found");
            byte[] sendData = ip.getBytes();
            DatagramPacket sendPacket = new DatagramPacket(sendData,
                sendData.length, receivePacket.getAddress(),
                receivePacket.getPort());
            socket.send(sendPacket);
            socket.close();
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

Question 8: Congestion Control

```
public class CongestionControl {
    public static void main(String[] args) {
        int cwnd = 1, ssthresh = 16;
        System.out.println("Round\tcwnd");
        for (int i = 1; i <= 20; i++) {
            System.out.println(i + "\t" + cwnd);
            if (cwnd < ssthresh) {
                cwnd = cwnd * 2;
            } else {
                cwnd = cwnd + 1;
            }
            if (i == 10) {
                System.out.println("Packet loss detected!");
                ssthresh = cwnd / 2;
                cwnd = 1;
            }
        }
    }
}
```

Question 9: Distance Vector Routing

```
import java.util.Scanner;

public class DistanceVectorRouting {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter nodes: ");
        int n = sc.nextInt();
        int[][] cost = new int[n][n];
        int[][] dist = new int[n][n];
        System.out.println("Enter cost matrix:");
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++) {
                cost[i][j] = sc.nextInt();
                dist[i][j] = cost[i][j];
            }
        for (int k = 0; k < n; k++)
            for (int i = 0; i < n; i++)
                for (int j = 0; j < n; j++)
                    if (dist[i][j] > cost[i][k] + dist[k][j])
                        dist[i][j] = cost[i][k] + dist[k][j];
        System.out.println("\nShortest distance matrix:");
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++)
                System.out.print(dist[i][j] + "\t");
            System.out.println();
        }
        sc.close();
    }
}
```

Question 10: Link State Routing (Dijkstra)

```
import java.util.*;

public class LinkStateRouting {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter nodes: ");
        int n = sc.nextInt();
        int[][] graph = new int[n][n];
        System.out.println("Enter cost matrix:");
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                if ((graph[i][j] = sc.nextInt()) == 0 && i != j)
                    graph[i][j] = 999;
        System.out.print("Enter source: ");
        int src = sc.nextInt();
        int[] dist = new int[n];
        boolean[] visited = new boolean[n];
        for (int i = 0; i < n; i++) {
            dist[i] = graph[src][i];
            visited[i] = false;
        }
        dist[src] = 0;
        visited[src] = true;
        for (int i = 0; i < n - 1; i++) {
            int min = 999, u = -1;
            for (int j = 0; j < n; j++)
                if (!visited[j] && dist[j] < min) {
                    min = dist[j];
                    u = j;
                }
            visited[u] = true;
            for (int j = 0; j < n; j++)
                if (!visited[j] && dist[u] + graph[u][j] < dist[j])
                    dist[j] = dist[u] + graph[u][j];
        }
        System.out.println("\nDistances from source:");
        for (int i = 0; i < n; i++)
            System.out.println("Node " + i + ": " + dist[i]);
        sc.close();
    }
}
```

Question 13: HTTP Web Client

```
import java.io.*;
import java.net.*;

public class HTTPClient {
    public static void main(String[] args) {
        try {
            Socket socket = new Socket("example.com", 80);
            PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
            out.println("GET / HTTP/1.1");
            out.println("Host: example.com");
            out.println("Connection: close");
            out.println();
            BufferedReader in = new BufferedReader(
                new InputStreamReader(socket.getInputStream()));
            String line;
            while ((line = in.readLine()) != null)
                System.out.println(line);
            socket.close();
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

Question 14: TCP Echo Server

```
import java.io.*;
import java.net.*;

public class TCPEchoServer {
    public static void main(String[] args) {
        try {
            ServerSocket serverSocket = new ServerSocket(1234);
            System.out.println("Server waiting...");
            Socket socket = serverSocket.accept();
            BufferedReader in = new BufferedReader(
                new InputStreamReader(socket.getInputStream()));
            PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
            String message = in.readLine();
            System.out.println("Received: " + message);
            out.println(message);
            socket.close();
            serverSocket.close();
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}

// CLIENT
import java.io.*;
import java.net.*;
import java.util.Scanner;

public class TCPEchoClient {
    public static void main(String[] args) {
        try {
            Socket socket = new Socket("localhost", 1234);
            BufferedReader in = new BufferedReader(
                new InputStreamReader(socket.getInputStream()));
            PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
            Scanner sc = new Scanner(System.in);
            System.out.print("Enter message: ");
            String message = sc.nextLine();
            out.println(message);
            String response = in.readLine();
            System.out.println("Echo: " + response);
            socket.close();
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

Question 15: UDP Echo Server

```
import java.net.*;

public class UDPEchoServer1 {
    public static void main(String[] args) {
        try {
            DatagramSocket socket = new DatagramSocket(9876);
            byte[] receiveData = new byte[1024];
            DatagramPacket receivePacket = new DatagramPacket(
                receiveData, receiveData.length);
            socket.receive(receivePacket);
            String message = new String(receivePacket.getData(), 0,
                receivePacket.getLength());
            byte[] sendData = message.getBytes();
            DatagramPacket sendPacket = new DatagramPacket(sendData,
                sendData.length, receivePacket.getAddress(),
                receivePacket.getPort());
            socket.send(sendPacket);
            socket.close();
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}

// CLIENT
import java.net.*;
import java.util.Scanner;

public class UDPEchoClient1 {
    public static void main(String[] args) {
        try {
            DatagramSocket socket = new DatagramSocket();
            Scanner sc = new Scanner(System.in);
            System.out.print("Message: ");
            String message = sc.nextLine();
            InetAddress serverAddress =
                InetAddress.getByName("localhost");
            DatagramPacket sendPacket = new DatagramPacket(
                message.getBytes(), message.length(), serverAddress, 9876);
            socket.send(sendPacket);
            byte[] receiveData = new byte[1024];
            DatagramPacket receivePacket = new DatagramPacket(
                receiveData, receiveData.length);
            socket.receive(receivePacket);
            System.out.println("Echo: " + new String(
                receivePacket.getData(), 0, receivePacket.getLength()));
            socket.close();
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

Question 16: TCP/UDP Performance

```
import java.util.Random;

public class TCP_UDP_Simulation {
    public static void main(String[] args) {
        Random rand = new Random();
        int packets = 10, tcpDelivered = 0, udpDelivered = 0;
        for (int i = 1; i <= packets; i++)
            tcpDelivered++;
        for (int i = 1; i <= packets; i++)
            if (rand.nextInt(100) < 70)
                udpDelivered++;
        System.out.println("TCP: " + tcpDelivered + "/" + packets);
        System.out.println("UDP: " + udpDelivered + "/" + packets);
        System.out.println("TCP%: " + (tcpDelivered*100/packets));
        System.out.println("UDP%: " + (udpDelivered*100/packets));
    }
}
```

Question 17: CRC Error Detection

```
import java.util.Scanner;

public class CRC {
    static String xor(String a, String b) {
        String result = "";
        for (int i = 1; i < b.length(); i++)
            result += (a.charAt(i) == b.charAt(i)) ? '0' : '1';
        return result;
    }

    static String modDiv(String data, String key) {
        int keyLen = key.length();
        String temp = data.substring(0, keyLen);
        int i = keyLen;
        while (i < data.length()) {
            if (temp.charAt(0) == '1')
                temp = xor(key, temp) + data.charAt(i);
            else
                temp = xor("0".repeat(keyLen), temp) + data.charAt(i);
            i++;
        }
        return temp;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Data: ");
        String data = sc.next();
        System.out.print("Generator: ");
        String key = sc.next();
        String appendedData = data + "0".repeat(key.length() - 1);
        String remainder = modDiv(appendedData, key);
        String transmitted = data + remainder;
        System.out.println("Transmitted: " + transmitted);
        System.out.print("Received: ");
        String received = sc.next();
        String check = modDiv(received, key);
        if (check.contains("1"))
            System.out.println("Error detected!");
        else
            System.out.println("No error.");
    }
}
```

Question 18: TCP String Processing

```
import java.io.*;
import java.net.*;

public class TCPServerUpper {
    public static void main(String[] args) {
        try {
            ServerSocket serverSocket = new ServerSocket(5000);
            Socket socket = serverSocket.accept();
            BufferedReader in = new BufferedReader(
                new InputStreamReader(socket.getInputStream()));
            PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
            String message = in.readLine();
            out.println(message.toUpperCase());
            socket.close();
            serverSocket.close();
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}

// CLIENT
import java.io.*;
import java.net.*;
import java.util.Scanner;

public class TCPClientUpper {
    public static void main(String[] args) {
        try {
            Socket socket = new Socket("localhost", 5000);
            PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
            BufferedReader in = new BufferedReader(
                new InputStreamReader(socket.getInputStream()));
            Scanner sc = new Scanner(System.in);
            System.out.print("Message: ");
            String message = sc.nextLine();
            out.println(message);
            System.out.println("Response: " + in.readLine());
            socket.close();
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

Question 19: Routing Protocol Performance

```
import java.util.Random;

public class RoutingPerformance {
    public static void main(String[] args) {
        Random rand = new Random();
        int packets = 10, dvDelivered = 0, lsDelivered = 0;
        for (int i = 1; i <= packets; i++)
            if (rand.nextInt(100) < 80)
                dvDelivered++;
        for (int i = 1; i <= packets; i++)
            if (rand.nextInt(100) < 95)
                lsDelivered++;
        System.out.println("DV: " + dvDelivered + "/" + packets);
        System.out.println("LS: " + lsDelivered + "/" + packets);
        System.out.println("DV%: " + (dvDelivered*100/packets));
        System.out.println("LS%: " + (lsDelivered*100/packets));
    }
}
```